

eRoBo 10

Collaborative Robot Platform

Complete Technical & Engineering Documentation

Project	erobo10_motion — 6-DOF ZeroErr eRob cobot (10 kg class, EtherCAT)
Document	Full project reference: architecture, motion, APIs, hardware, deployment
Version	eRoBo Platform 2.0
Date	13 July 2026
Prepared by	Aryan Singh — Robotics, Integra

Table of Contents

1. Introduction and System Overview	6
1.1 What eRoBo 10 is	6
1.2 Design pillars	6
1.3 Capabilities at a glance	6
1.4 How to read this document	7
2. System Architecture	8
2.1 The two planes	8
2.2 The processes	9
2.3 Command and state flow	9
2.4 Freedrive (hand-guiding) path	10
2.5 Startup order and handshake	10
3. The Shared-Memory Contract	11
3.1 One block, five regions	11
3.2 How Python "gets" the C++ logic	11
3.3 The seqlock	11
3.4 Command types	12
4. The Real-Time Control Plane	14
4.1 ecat_control_node — the hardware driver	14
4.2 control_node — the simulation twin	14
4.3 ik_solver — goal to joint target	14
4.4 motion_planner — trajectory generation	15
5. The Motion Library (core/include/erm)	16
6. The Kinematic Model	17
6.1 DH parameters	17
6.2 Forward and inverse kinematics	17
7. Dynamics, Torque and Feasibility	18
8. Motion Types	19
8.1 Palletizing	19
9. Tool Center Point (TCP) and the Tool Library	20
9.1 What a TCP is and why it matters	20
9.2 The mathematics	20

9.3 Vendor conventions	20
9.4 Calibration	20
9.5 How the active TCP reaches every move	21
9.6 The tool library	21
10. Waypoints, Programs and Play/Pause	22
10.1 Named waypoints	22
10.2 Programs	22
10.3 Play / pause (on-path feed-hold)	22
11. Robot Writing	23
11.1 Pipeline	23
11.2 Use	23
12. Writing Planes and Automatic Table-Plane Detection	24
12.1 Named writing planes	24
12.2 Auto-detect the table by touch probing	24
13. Contact Detection	25
13.1 Method	25
14. Safety System and Self-Collision	26
14.1 Three layers	26
14.2 Self-collision	26
14.3 Configuration and recovery	26
15. Joint Calibration	27
16. The Bridge Layer and Persistent Stores	28
17. The Production Controller (erobo_controld)	29
18. API Reference	30
18.1 WebSocket API (ws://robot:8765)	30
18.1.1 Motion and drive control	30
18.1.2 Programs, waypoints and application layer	31
18.1.3 Configuration and tooling	31
18.2 REST API (http://robot:8080/api/v1)	32
18.3 Modbus TCP (robot:1502, unit id 1)	32
18.4 MQTT (opt-in)	32
19. User Interfaces	33

19.1 erobo10_viewer.html — desktop 3-D viewer	33
19.2 cobot_mobile.html — phone UI	33
19.3 eRoBo Studio — Electron + React pendant	33
20. Hardware Reference — ZeroErr eRob Actuators	34
20.1 Joint to actuator map	34
20.2 Torque, speed, power, encoders	34
20.3 Brake, STO and freedrive	34
20.4 CiA-402 objects used	35
21. EtherCAT Integration	36
22. Build System	37
23. Deployment	38
24. Running the Stack	39
24.1 Simulation (no hardware)	39
24.2 Real robot	39
25. Testing and Verification	40
26. Recent Feature Additions	41
26.1 TCP application to programmed motion	41
26.2 The named tool library	41
26.3 Drive-fault popup	41
26.4 On-path feed-hold pause	41
26.5 Per-item files	41
26.6 Unique-name enforcement and item editing	41
27. Complete File and Directory Reference	42
27.1 Top-level layout	42
27.2 Root files	43
27.3 apps/ — real-time control plane	43
27.4 core/include/erm/ — header-only motion library	44
27.5 bridge/ — JSON to shared memory + persistent stores	45
27.6 controller/ — production controller	45
27.7 config/	45
27.8 robot/ — robot description and meshes	46
27.9 tools/ — front-ends and utilities	46

27.10 scripts/ — build and launch	46
27.11 deploy/ — installer, services, Docker	47
27.12 test/ — unit and integration tests	47
27.13 studio/ — eRoBo Studio (Electron + React + TypeScript)	48
27.14 run/ — runtime state (git-ignored)	49
27.15 docs/ — documentation set	49
28. Troubleshooting	50
29. Safety and Residual-Risk Notice	51
30. Glossary	52

1. Introduction and System Overview

1.1 What eRoBo 10 is

eRoBo 10 (`erobo10_motion`) is a complete, deployable platform for a **6-DOF collaborative robot** in the 10 kg payload class, built around **ZeroErr eRob** all-in-one servo actuators driven over **EtherCAT**. The platform spans the entire stack: a real-time C++ motion engine, a configurable safety system, a production controller exposing WebSocket / REST / Modbus / MQTT interfaces, persistent stores for tools, waypoints and programs, and multiple operator interfaces (a browser 3-D viewer, a phone UI, and the Electron/React **eRoBo Studio** pendant).

The system takes a high-level intent from an operator interface — "move the tool tip to this pose", "jog this joint", "hand-guide the arm", "run this program", "write this text on the paper" — and turns it into smooth, jerk-limited, torque-feasible motion that drives six servo drives in real time, while streaming measured state back to the interface as a live digital twin.

1.2 Design pillars

- **Deterministic real-time core, ROS-free.** The control plane is C++ with lock-free shared-memory IPC (the way commercial cobot controllers are built). External systems integrate through documented network APIs; a ROS 2 driver could sit on the WebSocket interface exactly as vendor ROS drivers do.
- **One validated motion library.** A header-only, dependency-free library (`core/include/erm/`) provides URDF-exact forward kinematics (validated to $\sim 1.7 \mu\text{m}$), a no-fail Levenberg-Marquardt inverse-kinematics solver, jerk-limited S-curve trajectory generation for PTP / LIN / MoveP / MoveC, RNEA inverse dynamics with payload, sensorless contact detection, self-collision prediction, and the safety monitor.
- **Correctness proven by tests.** Unit suites (FK, IK, trajectory, safety, TCP) plus a 40+ check full-stack integration test. IK converges on 3000/3000 random poses to under $1 \mu\text{m}$; the Jacobian matches finite differences to $2.4\text{e-}10$; MoveL in-loop accuracy is $\leq 0.2 \text{ mm}$ in simulation.
- **Safety in depth.** A hardware E-STOP / STO chain that is independent of all software, a 1 kHz measured-state safety monitor, and planner-side limiting that keeps the arm away from the limits so the monitor rarely has to act.

1.3 Capabilities at a glance

- Motion primitives: **MoveJ** (joint), **MoveJ-to-pose / PTP** (IK once), **MoveL** (Cartesian straight line), **MoveP** (constant tool-speed blended path), **MoveC** (circular arc / full circle), **move_rel** (relative offset), velocity **jog** (joint / base / tool frame), and **freedrive** (hand-guiding).
- Tooling: full **Tool Center Point** layer with a numeric editor and 4-point + orientation teach wizards, and a persistent **tool library** (save / apply / edit / rename / save-as / delete, unique names, per-item files).
- Applications: teachable **named waypoints**, a **program** editor and completion-gated executor with **play/pause** (on-path feed-hold), **palletizing**, **robot writing** (text $\rightarrow$ single-stroke toolpaths), **writing planes** with 3-point teach, and **automatic table-plane detection by touch probing**.

- Monitoring and safety: sensorless **contact detection**, **self-collision** prediction, a configurable safety system (speed / force / reach / planes / zones), live per-joint **telemetry** (temperature, current, voltage, torque, brake), and singularity awareness.
- Integration: WebSocket + REST APIs, Modbus TCP slave for PLCs, optional MQTT publisher, users / roles / sessions, a SQLite event log / flight recorder, and backup / restore.

1.4 How to read this document

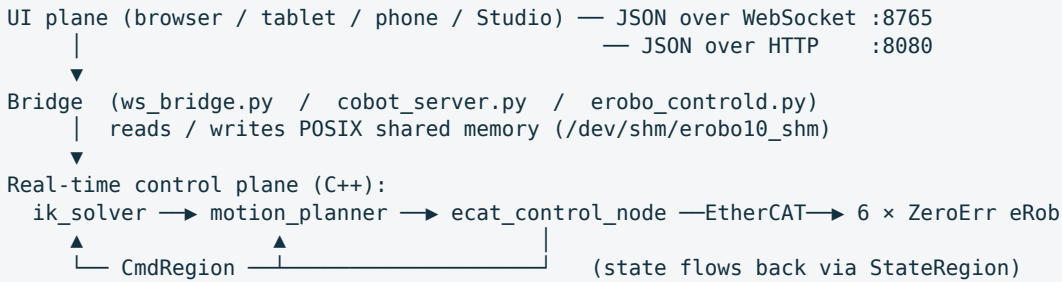
The document is ordered from the inside out. It begins with the architecture and the shared-memory contract that binds the system together, then covers each real-time process, the motion mathematics, and the higher-level features (TCP, waypoints/programs, writing, planes, contact, safety, calibration). It then documents the bridge and controller services, the complete API surface, the operator interfaces, the actuator hardware, the EtherCAT layer, and finally build / deployment / testing, the most recent feature additions, a complete file reference, troubleshooting, and a glossary.

Units are SI throughout unless stated otherwise: metres, radians, radians/second, newton-metres, kilograms. Orientation is given as a quaternion [w, x, y, z] or roll-pitch-yaw [r, p, y] in radians.

2. System Architecture

2.1 The two planes

The system is split into a non-real-time **UI plane** and a real-time **control plane**, joined by a translation **bridge**:



- **UI plane** — not real time. Buttons and joysticks emit small JSON commands and render the streamed state, including a Three.js 3-D twin. It never touches the drives directly. The only network boundary in the whole system is UI ⇌ bridge.
- **Bridge** — translates JSON ⇌ shared memory. It attaches to the same shared block the control plane uses, so bridge ⇌ control-plane communication is lock-free and microsecond-fast (no network in the control path).
- **Control plane** — C++ processes running at 1 kHz. It owns all the motion mathematics and is the only thing that talks to hardware.

2.2 The processes

Process	Language	Rate	Role
ecat_control_node	C++	1000 Hz	Owns the shared block; runs the EtherCAT master and CiA-402 drive state machine; streams setpoints to the drives; publishes measured state and telemetry
control_node	C++	1000 Hz	The simulation twin of the above — same shared-memory role, but an internal drive model instead of hardware
motion_planner	C++	1000 Hz	Jerk-limited S-curve trajectory generation; MoveJ/L/P/C; jog; stop; speed override; zone scaling; torque de-rate; feed-hold pause
ik_solver	C++	~500 Hz	Turns UI goals into joint targets: Movejoint passthrough, PTP solves IK once, LIN/P/C validate reachability
ws_bridge.py / cobot_server.py / erobo_controld.py	Python	50 Hz	JSON ⇄ shared memory; command dispatch; state broadcast; persistent stores; APIs

The processes do not call one another's functions. They share **one memory-mapped file** and coordinate purely by reading and writing fixed byte offsets under a seqlock.

2.3 Command and state flow

A motion goal takes the long path through the mathematics:

```
UI: {"cmd":"move_ptp","pos":[...],"rpy":[...],"speed":0.5}
→ bridge writes CmdRegion
→ ik_solver      reads CmdRegion → solves IK → writes IkRegion (target_q) + acks
→ motion_planner reads IkRegion → plans a jerk-limited, torque-feasible trajectory
                  → streams q_cmd into SpRegion at 1 kHz
→ ecat_control_node reads SpRegion → CSP/CSV control law → EtherCAT → the six drives move
→ ecat_control_node publishes StateRegion → bridge → UI updates the 3-D twin and readouts
```

Direct controls are shorter. enable, disable, estop, reset, freedrive, and set_payload are read directly by ecat_control_node (e-stop and reset latch there). jog and stop are read by motion_planner, which integrates a jerk-limited velocity follower into the setpoint stream so jogging is as smooth as a planned move. State always flows the other way: the control node writes StateRegion at 1 kHz, the bridge samples it (about 50 Hz for the WebSocket broadcast) and the UI renders it.

2.4 Freedrive (hand-guiding) path

Freedrive is special because a velocity servo holding zero velocity resists being pushed. The flow is:

```
UI "Free-drive" → {"cmd":"enable"} + {"cmd":"freedrive","on":true}
→ ecat_control_node: freedrive implies enable (brakes release);
    switches each drive mode-of-operation 0x6060 from CSV(9)/CSP(8) to CST(10, torque)
    via a runtime SDO, and commands zero torque (0x6071 = 0) → the arm is back-drivable
→ motion_planner: while mode == FREEDRIVE it sets q_cmd = measured q (tracks your hand)
→ "Lock" / {"cmd":"freedrive","on":false}: drives switch back, hold the current pose (no snap-back)
```

The PDO map is unchanged by this — only 0x6060 is toggled at runtime — so normal motion cannot be affected. Because zero-torque freedrive is held only by gear friction, the arm can sag under gravity; support it when engaging. The hardware E-STOP / STO must remain independent of software.

2.5 Startup order and handshake

The control node starts first: it zeroes the shared block, seeds the setpoint to the home pose, writes an initial state, then publishes a magic number (ER01) last. Every other process maps the file and spins until the magic appears before using it, so no one ever reads an uninitialised block. The planner additionally waits for the state sequence to advance (the control node has written at least once) before seeding its commanded position from the measured state. The run scripts launch the processes in the correct order.

3. The Shared-Memory Contract

3.1 One block, five regions

The single source of truth between the bridge and the control plane is one mmap-backed block at `/dev/shm/erobo10_shm`, defined by struct `ShmBlock` in `core/include/erm/shm_layout.hpp` and mirrored byte-for-byte in Python by `bridge/shm_layout.py`. On Linux `/dev/shm` is a RAM-backed tmpfs, so this is pure shared RAM — no disk, no network, no serialization.

The block has five regions, each with exactly one writer (so there is never write contention) and its own sequence counter:

Region	Written by	Read by	Carries
CmdRegion	bridge	ik_solver, motion_planner, control node	latest command: type, target_q / target_pose, speed, accel, enable / estop / reset, payload, TCP transform, contact settings
IkRegion	ik_solver	motion_planner	solved goal: move type, target_q or target_pose, reachability, gravity utilisation
PathRegion	bridge	ik_solver, motion_planner	MoveP / MoveC waypoint list + blend + tool speed / accel
SpRegion	motion_planner	control node	the 1 kHz streamed setpoint: q_cmd, qd_cmd, moving, progress, speed_scale
StateRegion	control node	bridge, motion_planner, ik_solver	measured q, qd, torque, flange pose, TCP pose, manipulability, mode, enabled / estop, per-joint status, telemetry, collision / contact fields

3.2 How Python "gets" the C++ logic

There is no RPC and no binding. The C++ side defines a packed struct (`#pragma pack(1)`) laid out at fixed byte offsets; the control node mmmaps a file of that exact size. Python declares the same struct with ctypes and `_pack_ = 1`, so its field offsets are byte-identical (verified at import by `assert ctypes.sizeof(...) == ...`), and overlays it on the same file with `ShmBlock.from_buffer(mm)`. When Python writes `blk.cmd.target_q[0] = 0.3` and C++ reads `blk->cmd.target_q[0]`, they touch the same eight bytes of RAM. The "protocol" is the struct layout itself. A layout drift between the two languages is caught immediately by the size asserts.

3.3 The seqlock

A reader must never see a half-written frame. Instead of a mutex (which risks blocking or priority inversion in a 1 kHz loop) the system uses a **seqlock**:

```

WRITE (one writer):  seq |= 1          (odd = "writing")    ← release fence
                    ...write fields...                  ← release fence
                    seq += 1          (even = "done")
READ (any reader):  loop:
                    s1 = seq; if (s1 is odd) retry
                    ...copy fields out...
                    s2 = seq; if (s1 == s2) success else retry

```

If a write starts mid-read, $s2 \neq s1$ and the reader retries; readers never block the writer, so the 1 kHz loop stays deterministic. The helpers are `seqWrite` / `seqRead` in `core/include/erm/shm.hpp`, mirrored in Python by `ShmClient`. New commands are detected by the sequence bump: each writer increments its `*_seq` and consumers remember the last value and act only when it changes — cross-process "a new message arrived" signalling with no events or queues.

3.4 Command types

`CmdType` in `shm_layout.hpp` enumerates every command the control plane understands. Adding a value is layout-safe (it does not resize any struct):

Value	Name	Meaning
1	CMD_MOVE_JOINT	synchronized jerk-limited joint move (target_q)
2	CMD_MOVE_PTP	IK once, then joint move (target_pose)
3	CMD_MOVE_LIN	straight-line Cartesian move (target_pose)
4	CMD_STOP	smooth cosine-eased decel to rest
5	CMD_HOLD	hold current setpoint
6	CMD_JOG	velocity jog (frame + velocities in target_q)
7	CMD_MOVE_P	process move: PathRegion waypoints, constant speed + blends
8	CMD_ENABLE	power-enable the drives (CiA-402 → Operation Enabled)
9	CMD_DISABLE	power-disable (hold, brakes engaged)
10 / 11	CMD_FREEDRIVE_ON / _OFF	enter / leave hand-guiding
12	CMD_SET_PAYLOAD	set tool / payload mass for the torque model
13	CMD_MOVE_C	circular arc (PathRegion via + end)
14	CMD_ECOT_RESET	restart the EtherCAT connection: clear faults, re-bring-up slaves
15	CMD_SET_TCP	set the tool-center-point transform + payload + CoG
16	CMD_SET_COLLISION	enable / disable the self-collision monitor
17	CMD_SET_CONTACT	external-contact detector: sensitivity / on / test

Value	Name	Meaning
18	CMD_SET_SIGN	set per-joint direction sign (+1 / -1) live
19	CMD_PAUSE	feed-hold: decelerate to a stop on the path and hold (resumable)
20	CMD_RESUME	release a feed-hold: ramp velocity back up along the same trajectory

Values 19 and 20 are the on-path pause / resume added for program feed-hold (documented later). Other real-time nodes safely ignore command types they do not handle, so introducing a new type never disturbs the existing ones.

4. The Real-Time Control Plane

The control plane is four C++ roles (one with a real and a simulated variant). Each is a thin `main()` over the header-only `erm::` library. All run at a hard 1 kHz using `std::this_thread::sleep_until` on a steady clock; on hardware the loop runs under `SCHED_FIFO` with `mlockall` and EtherCAT distributed-clock sync.

4.1 `ecat_control_node` — the hardware driver

`apps/ecat_control_node.cpp` is the only process that talks to hardware. Responsibilities:

- **EtherCAT bring-up.** Brings up the IgH EtherCAT master, configures the six ZeroErr drives, maps the PDOs, and (by default) enables Distributed Clocks (1 ms SYNC0) for tight synchronisation. The `--no-dc` flag disables DC; `--shm` sets the block path.
- **CiA-402 state machine.** Drives each slave `FAULT` → `fault-reset` → `shutdown` → `switch-on` → `enable` → `Operation Enabled`. Fault reset is generated as a true rising edge (a constant reset word can never clear a re-latched fault). Transient faults auto-recover on a strike budget; persistent faults latch and are reported with the drive error code.
- **Setpoint streaming.** Reads `SpRegion.q_cmd` and writes the drive target each cycle. The drives run **CSP** (Cyclic-Sync Position, mode 8): the commanded joint angle is converted to encoder counts and written to object 0x607A; the drive closes the position loop and stops exactly. A per-cycle setpoint sanity clamp (2× rated speed) guarantees no upstream fault can command a jump.
- **Encoder ↔ joint mapping.** $\theta = \text{JOINT_SIGN} \cdot (\text{counts} / 2^{19} \cdot 2\pi) + \text{JOINT_OFFSET}$. This node is the only place that maps between encoder counts and joint angle; everything above (IK, planner, viewer) works in joint-angle space. Sign and offset are loaded from `run/calibration.txt` at startup (no rebuild to change them).
- **Freedrive.** Switches each drive's mode-of-operation to CST (torque) via a runtime SDO and commands zero torque, making the arm back-drivable, then switches back and holds on exit.
- **Telemetry.** Publishes per-joint temperature (0x22A2), current (0x6078), DC-link voltage (0x6079), measured torque (0x6077, cyclic PDO), and brake state (0x4602). Everything except torque is read off the real-time path via async SDO (polled slowly) so the 1 kHz servo loop is never stalled.
- **Safety hooks.** Runs the self-collision monitor and the contact detector; on a critical trip it disables the drives (protective stop), publishing the cause. It also publishes the TCP pose (`tcp_pos`, `tcp_quat`) computed from the active tool transform.

4.2 `control_node` — the simulation twin

`apps/control_node.cpp` fills the same shared-memory role but drives an internal `SimDrives` model instead of hardware, so the whole stack runs on a laptop with no robot. It runs the identical FK, manipulability, safety and contact logic; the "measured" torque is the model plus friction (plus an optional injected test torque) so contact detection can be exercised offline. A virtual table can be enabled with the environment variable `ERoBo_SIM_TABLE_Z=<z>` so that auto-plane touch probing runs end to end in simulation.

4.3 `ik_solver` — goal to joint target

`apps/ik_solver.cpp` consumes UI goals from `CmdRegion` and produces joint targets in `IkRegion`:

- **CMD_MOVE_JOINT** — a limit-checked passthrough (already joint space).
- **CMD_MOVE_PTP** — solves IK once, warm-started from the current joint state; if the warm start is infeasible or the gravity-hold torque is too high, it tries comfortable alternative seeds and keeps the lightest torque-feasible configuration (as UR/ABB pick a posture).
- **CMD_MOVE_LIN** — validates reachability of the target and forwards the pose to the planner for per-step IK; it also runs a straight-line feasibility pre-check that rejects a line crossing a singularity.
- **CMD_MOVE_P / CMD_MOVE_C** — validate every waypoint of the path (per-waypoint IK) before the planner runs it.

The solver applies the active **TCP**: $\text{solveIKtcp}(P) = \text{solveIK}(P \cdot T_{\text{tool}}^{-1})$, so Cartesian targets are interpreted as tool-tip targets. A **torque-feasibility gate** refuses a target whose gravity-hold torque exceeds ~98 % of any joint's effort, returning a clear reason instead of running it into a drive fault. The solver deliberately does not act on command types it does not recognise (for example **CMD_PAUSE**), so a pause never triggers a re-plan.

4.4 motion_planner — trajectory generation

`apps/motion_planner.cpp` is the 1 kHz jerk-limited (S-curve) trajectory generator. It reads `IkRegion` goals and streams C^2 -continuous setpoints into `SpRegion`:

- **MoveJ** — synchronized 7-phase double-S profile per joint, time-scaled so all joints finish together.
- **MoveL / MoveP / MoveC** — a jerk-limited scalar advances along the Cartesian path (position interpolation + quaternion SLERP for lines, a 3-point circumcircle for arcs, tangent-blended segments for MoveP), solving IK for the interpolated **tool-tip** pose every 1 ms (warm-started from the previous setpoint).
- **Jog** — a per-joint jerk-limited velocity follower; for base / tool frame it converts the Cartesian twist to joint velocity via the damped-least-squares Jacobian. A 0.25 s watchdog ramps to zero if jog messages stop (dead-client safety).
- **Global speed override and safety zones** — a live speed-override scale (`ovr_cur`) and a zone scale (`safe_scale`) are slew-limited and multiplied into the trajectory time advance, so speed changes and reduced zones are bump-free. Entering a forbidden boundary triggers a cosine-eased decel.
- **Torque-feasible time-scaling** — a move whose peak inverse-dynamics torque would exceed effort is automatically slowed until it fits.
- **Singularity handling** — Cartesian moves cap per-cycle joint velocity at the joint limit and time-scale the path, so the arm slows through a near-singular region instead of demanding impossible joint speed. A discontinuity guard (abort on an IK jump) and a flange-tracking guard (abort if the achieved TCP drifts more than 5 cm from the commanded path) stop an infeasible Cartesian move cleanly rather than flailing.
- **Bump-free engage.** While the drives are disabled or in freedrive, the planner tracks the measured position, so there is no following-error jump when drives engage.
- **Feed-hold pause.** A `pause_scale` ramps the trajectory velocity smoothly to zero on **CMD_PAUSE** (freezing trajectory time so the arm holds exactly on its path) and back to one on **CMD_RESUME**, so a program can be paused and resumed mid-move without leaving the path.

5. The Motion Library (core/include/erm)

The motion engine is a header-only, dependency-free C++ library in the `erm::` namespace. Because it has no external dependencies it ports cleanly to a real-time control box, and each `apps/*.cpp` is a small `main()` that includes these headers.

Header	Contents
<code>linalg.hpp</code>	SE(3)/SO(3) math: Vec3, Mat3, Mat4, quaternions, rotation vectors, SLERP, SO(3) log/exp, a dense linear solver
<code>kinematics.hpp</code>	Forward kinematics and the geometric Jacobian over the DH model; damped-least-squares; manipulability
<code>erobo10_dh.hpp</code>	Auto-generated, URDF-validated Denavit-Hartenberg table + per-joint limits (position, velocity, accel, jerk, effort)
<code>erobo10_dyn.hpp</code>	Auto-generated per-joint frames + link rigid-body inertials (mass, centre of mass, inertia)
<code>dynamics.hpp</code>	Recursive Newton-Euler inverse dynamics $\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$, with optional flange payload
<code>ik.hpp</code>	Robust full-6-DOF inverse kinematics: Levenberg-Marquardt damped least squares with quaternion error, joint-limit clamping, warm start and multi-seed restarts
<code>trajectory.hpp</code>	Jerk-limited S-curve profiles, synchronized PTP (JointTrajectory), Cartesian CartesianTrajectory, BlendedPath (MoveP), CircularTrajectory (MoveC)
<code>tcp.hpp</code>	The tool-center-point frame layer: tcpFK, solveIKtcp, tool transform from pos+quat / rpy / rotnvec
<code>contact.hpp</code>	Sensorless external-torque contact detector (generalized-momentum-style residual)
<code>collision.hpp</code> , <code>erobo10_collision_geom.hpp</code>	Capsule self-collision model and geometry
<code>safety.hpp</code>	The measured-state safety monitor (speed / force / reach / planes / zones)
<code>shm.hpp</code>	POSIX shared-memory map/unmap + the seqlock read/write helpers
<code>shm_layout.hpp</code>	The canonical shared-memory contract: the five regions, CmdType, sizes, magic / version

`hal.hpp` carries the EtherCAT hardware-abstraction shape shared by the nodes. The old `ecat_hal.hpp` / `ecat_config.hpp` are tombstones — the EtherCAT layer and its tunables now live at the top of `apps/ecat_control_node.cpp`.

6. The Kinematic Model

6.1 DH parameters

The Denavit-Hartenberg table was extracted from `robot/erobo10.urdf` and validated against the exact URDF forward kinematics: maximum position + orientation error **1.78e-6 m** over 50,000 random configurations across the full joint range (that residual is dominated by the URDF rounding $\pi/2$ to 1.5708; the DH table uses exact angles). Convention (standard / distal): $T_i = R_z(\theta_i + q_i) \cdot T_z(d_i) \cdot T_x(a_i) \cdot R_x(\alpha_i)$, with the base frame identity and the joint-1 axis on the base z-axis.

Joint	a (m)	α (rad)	d (m)	θ_0 (rad)	limit (rad)	v_max (rad/s)	$\tau_{\max}$ (N·m)
1	0.000001	1.570799	0.123000	0.807906	[-3.140, 3.140]	1.57	140
2	0.611996	3.141585	0.000000	1.583038	[-3.140, 3.140]	1.57	110
3	0.590001	-0.000007	0.000000	-0.219999	[-3.140, 3.140]	1.57	110
4	0.000000	1.570794	-0.128600	1.547938	[-3.140, 3.140]	1.57	107
5	0.000003	1.570795	0.098000	3.111059	[-3.140, 3.140]	1.57	87
6	0.000000	0.000000	0.000000	0.000000	[-3.140, 3.124]	1.57	57

Structure: J1 is the vertical base yaw; J2-J3-J4 are parallel (a planar arm — upper arm $a_2 = 0.612$ m, forearm $a_3 = 0.590$ m in the DH frame); J5 and J6 are perpendicular wrist axes. The tool / flange offset from DH frame 6 to the URDF link 6 is a translation $z = -0.0785$ m with rotation $r_{py} = [\pi, 0, -1.308]$ rad. Accel / jerk limits (tuned for a 10 kg payload, conservative and tunable): $a_{\max} = [6, 5, 6, 8, 8, 10]$ rad/s², jerk = [120, 100, 120, 160, 160, 200] rad/s³.

The $\tau_{\max}$ values shown are the original URDF efforts. On real hardware the per-joint torque ceilings are set to the eRob avg-load-max for the fitted gear ratios — {281, 281, 140, 51, 51, 51} N·m for J1..J6 (see the Hardware Reference). The dynamics gate and de-rate use those hardware values.

6.2 Forward and inverse kinematics

Forward kinematics is the DH product times the tool transform; the geometric Jacobian and Yoshikawa manipulability come from the same model. Inverse kinematics (`ik.hpp`) is Levenberg-Marquardt damped least squares on the 6-D pose error (position + quaternion), with joint-limit clamping, warm start from the current configuration, and multiple seed restarts so it never fails to converge on a reachable pose. Validation: 3000/3000 random poses converge to under 1 μ m; the Jacobian matches finite differences to 2.4e-10; warm-started per-cycle IK costs about 2 μ s mean, 21 μ s worst — well inside the 1 ms budget, so MoveL / MoveP can close IK every tick.

7. Dynamics, Torque and Feasibility

The planner originally used only kinematic limits (velocity / accel / jerk) and never checked joint torque, which is unsafe at load: holding the home pose with a 10 kg payload needs about 128 N·m at joint 2, and an extended shoulder pose about 172 N·m — both beyond the joint limit, which would fault the drive or sag the arm. The dynamics layer models this:

- **Inverse-dynamics model** (`dynamics.hpp`, RNEA over the URDF rigid bodies): $\tau = M(q) \cdot \ddot{q} + C(q, \dot{q}) \cdot \dot{q} + G(q)$ with an optional flange point-mass payload at a centre of gravity. Gravity torque was cross-validated against an independent computation to 0.1 N·m.
- **Feasibility gate** (`ik_solver`): a target whose gravity-hold torque exceeds ~98 % of any joint's effort is refused with a clear reason instead of being run into a fault.
- **Torque-feasible speed de-rate** (planner): a move whose peak inverse-dynamics torque would exceed effort is automatically slowed until it fits (verified: a full-speed MoveJ is 39 % of effort at 0 kg, 118 % at 5 kg and 197 % at 10 kg, both de-rated).
- **Payload**: `{"cmd": "set_payload", "kg": X, "cog": [...]}` sets the mass and CoG used by the gate, de-rate and telemetry; it is sticky across commands. Setting a TCP also carries its payload and CoG.
- **Live telemetry**: state carries per-joint estimated torque (N·m) and `torque_util` (peak fraction of effort), shown green / amber / red in the interfaces.

8. Motion Types

Definitions follow Universal Robots' documentation (the de-facto cobot standard; ABB MoveJ/L/C and KUKA PTP/LIN/CIRC are equivalent). All are jerk-limited S-curves for smoothness.

Type	Meaning	Command	Engine
MoveJ (joints)	joints rotate directly to target angles; TCP traces a curve; no singularity problem; fastest	move_joint	JointTrajectory
MoveJ (to a pose)	IK once, then a joint move	move_ptp	IK → JointTrajectory
MoveL	TCP moves in a straight line; IK solved every interpolated point	move_lin	CartesianTrajectory
MoveP	constant tool speed with circular blends rounding corners; does not stop at waypoints	move_p	BlendedPath
MoveC	arc through current → via → end (3-point circumcircle); full:true sweeps 360°	move_c	CircularTrajectory
Relative	nudge the TCP by a delta in the base or tool frame (straight line)	move_rel	CartesianTrajectory

move_p takes up to 16 waypoints; the blend distance d sets the corner radius $R = d \cdot \tan(\beta/2)$ (smaller = sharper, larger = smoother) and the path is C1-continuous. jog continues only while messages arrive (0.25 s watchdog); frame joint = six joint rad/s, base / tool = a twist $[v_x, v_y, v_z, w_x, w_y, w_z]$. All Cartesian moves carry the two path-tracking guards described in the planner section, so an infeasible straight or round path stops cleanly (fall back to MoveJ or split into shorter segments).

8.1 Palletizing

Teach a **PICK** pose and a pallet **CORNER** pose as waypoints, then give a grid (counts $n_x \cdot n_y \cdot \text{layers}$, pitch dx, dy, dz , and an approach clearance). Each cell is computed as an offset from the corner and expands to pick-approach → PICK → retract → place-approach → PLACE(cell) → retract (transits are PTP, the vertical legs MoveL). The whole thing runs as a normal program, so each move is completion-gated and the collision / singularity safety applies; order:snake alternates row direction to cut travel. Implemented in bridge/palletizing.py, run via pallet_run or the viewer's Motion+ panel.

9. Tool Center Point (TCP) and the Tool Library

9.1 What a TCP is and why it matters

A TCP is a rigid transform **T_{flange→tcp}** (a translation + rotation from the tool output flange) plus the payload the tool carries (mass + centre of gravity). Native robot kinematics describe the bare **flange**; you care about the tool's working point — a pen tip, gripper fingers, weld nozzle or camera. Once a TCP is set, the whole stack works at the tool tip: the pose readout reports the tip, Cartesian targets are interpreted as tip targets, MoveL / MoveC move the tip in a straight line / true arc, and the payload feeds the torque model. Default (no tool) is identity = the bare flange, so existing behaviour is unchanged until a TCP is set.

9.2 The mathematics

The whole layer is two operations, inverses of each other (core/include/erm/tcp.hpp):

```
tcpFK(q)      = forwardKinematics(q) · T_tool      // report the tool tip
solveIKtcp(P) = solveIK( P · T_tool-1 )          // accept tool-tip targets
```

You are told where the tip must go, but IK only speaks flange, so you back off by the tool to get the flange target, then solve in flange space. The validated flange FK / IK are untouched — TCP is a pure pre/post multiply, so it is provably correct and all existing safety (singularity, IK-fail, collision, torque gate) still wraps every move. A zero / near-zero quaternion is treated as identity ("no tool" = flange).

Consequences: a translation-only TCP (a pen along +Z) makes wrist reorientation pivot the tip around an arc of radius equal to the tool length; a TCP with rotation redefines the tool's own axes so "MoveL in tool +Z" follows the tool axis. Move speed and the reorientation pivot are defined at the TCP.

9.3 Vendor conventions

The transform is universal; the encoding differs and mixing them up is the classic bug. Universal Robots use a 6-vector [x,y,z,Rx,Ry,Rz] where the rotation is an axis-angle / rotation vector; ABB tooldata uses a quaternion and bundles the payload; KUKA \$T00L uses ZYX Euler A B C. This project stores [x,y,z, qw,qx,qy,qz] (position + quaternion) in cmd.tcp[7] in shared memory, with payload mass and CoG alongside. The UI accepts roll-pitch-yaw degrees or UR axis-angle degrees and converts to the quaternion.

9.4 Calibration

- **4-point position teach.** Jog the tool tip onto one fixed reference point from four or more different orientations and capture each flange pose. Because the tip is at the same world point every time, the unknown offset $\mathbf{o}$ (in the flange frame) satisfies $(\mathbf{R}_i - \mathbf{R}_j) \cdot \mathbf{o} = \mathbf{t}_j - \mathbf{t}_i$; stacking all pairs gives an over-determined linear system solved by least squares, with the RMS residual reported. More points and more orientation spread improve conditioning; the solver recovers a synthetic offset to 0 m.
- **Orientation teach (point-along-axes).** Keep the flange orientation fixed and translate the tip along the tool +Z (and optionally +X); the recorded motion directions define the tool axes in the flange frame, orthonormalised and converted to a quaternion (roll is left

free for a round marker if +X is skipped).

- **Numeric.** Enter the offset x,y,z (mm), orientation (rpy deg or axis-angle deg), payload mass (kg) and CoG (mm), then Apply.

9.5 How the active TCP reaches every move

Applying a TCP sets one runtime tool transform that travels with every command in the shared-memory `cmd.tcp` field. `ik_solver` uses `solveIKtcp` for PTP / LIN / P / C; `motion_planner` interpolates the tool-tip path via `tcpFK`; the control node publishes the tip pose in state. Move-type semantics match UR/ABB, where a waypoint is fundamentally a TCP pose: MoveL / MoveP / MoveC always honour the active TCP; for a stored waypoint replay, when a tool is applied and the waypoint carries a taught pose, the step is replayed as PTP to the taught tool-tip pose (IK in TCP space) so that applying or changing the TCP actually changes where the arm lands — with no tool, or a joint-only waypoint, exact joint replay is preserved. So after Apply TCP, joint / PTP / linear / arc moves and both single-waypoint "Go" and full programs move the tool tip to the taught location.

9.6 The tool library

The controller keeps a persistent, named tool library (`bridge/tcp_config.py`, `ToolLibrary`, `run/tools.json`) alongside the single active TCP (`run/tcp_config.json`). Selecting a tool copies its record into the active store and applies it. The library supports:

- **Save current** — save the tool defined in the editor as a named config. Names must be unique (case-insensitive); a duplicate is rejected with a message.
- **Apply** — load a saved tool onto the robot (it becomes the active TCP, shared across UIs and persisted).
- **Edit** — load a tool into the editor, change values, and save changes in place (same name).
- **Rename** — change a tool's name (the per-item file is renamed and, if it was active, the active TCP is kept in sync).
- **Save as** — create a copy of a tool under a new name.
- **Delete** — remove a tool.

Each tool is also written as an individual JSON file in `run/tcp_configs/<name>.json`, created on save, moved on rename, copied on save-as, and removed on delete (the `tools.json` library file is kept intact as well).

10. Waypoints, Programs and Play/Pause

10.1 Named waypoints

Waypoints are persistent, named, joint-space poses shared across all interfaces (`bridge/waypoints.py`, `WaypointStore`, `run/waypoints.json`). Each waypoint stores an id, name, the six joint angles, an optional Cartesian pose (captured at the tool tip), and a creation time. Every operation re-reads the file so the WebSocket bridge and the HTTP server stay in sync. Operations: save current pose, go to a waypoint, edit its values, rename, save-as (copy), and delete. Names are unique (case-insensitive) — a duplicate save or rename is rejected with a message. Each waypoint is also written as `run/waypoints/<name>.json`, created on save, moved on rename, copied on save-as, and removed on delete.

10.2 Programs

A program is an ordered sequence of steps. The waypoint-program executor (`_run_program` in the bridge) runs each step gated on the previous motion completing, so steps never overlap. Step types: **j** (joint move / stored-waypoint replay), **l** (straight line to a pose), **p** (PTP to a pose via IK), and **c** (circular arc, where this waypoint is the via and the next is the end). The executor honours the active TCP (a "j" step with a taught pose and a tool applied is driven as PTP to the tool-tip pose), emits progress events (`start`, `step_start`, `step_done`, `loop`, `done`, `error`, `aborted`), and supports looping with a cycle counter. The production controller additionally has a typed-step program store (`bridge/programs.py`) with motion / I/O / tool / speed / payload steps.

10.3 Play / pause (on-path feed-hold)

Programs can be paused mid-motion and resumed on the same path. Pressing pause sends `CMD_PAUSE`; the planner ramps a `pause_scale` smoothly to zero, which freezes trajectory time so the arm decelerates along its path and holds exactly where it is (the move stays active, not completed). Resume sends `CMD_RESUME` and the scale ramps back to one, continuing the same trajectory to its waypoint. This is the industry-standard method (runtime velocity scaling of a time-parameterized trajectory, as in UR feed-hold and Movelt), and it keeps the tool on its programmed path rather than replanning. While paused, the bridge freezes the per-step completion timeout so a long hold never errors the step; a stop clears any hold before decelerating; starting a run clears any prior hold. The operator interface exposes this as a single Run / Pause / Resume button (Run when idle, Pause while running, Resume while held) with a separate Stop.

The deceleration is a smooth on-path ramp (about 0.3 s), not an instantaneous freeze — an instant stop would demand infinite deceleration. The arm halts as quickly as is safe while staying on the path, and resume ramps back up along the same trajectory. The independent hardware E-STOP / STO remains the emergency stop.

11. Robot Writing

The writing feature is a KUKA-style "robot writing" pipeline (text → vector strokes → Cartesian tool-paths → execute) built on the MoveL / MoveP + TCP layer. The marker configured as the TCP traces the letters.

11.1 Pipeline

```
text → single-stroke vector font (bridge/writing.py, Hershey futural.jhf)
      → laid out, scaled to a letter height in a 2-D writing plane
      → placed on a 3-D writing plane (a taught TCP pose / paper origin)
      → drawn at the marker TCP: pen-up travel (PTP) → pen-down (MoveL)
      → stroke (blended MoveP) → pen-up (MoveL), each completion-gated
```

- **Font.** A Hershey single-stroke face (bridge/fonts/futural.jhf; Allen Hershey 1967 — the pen-plotter standard used by AxiDraw / Inkscape Hershey Text). Single-stroke centrelines (not TrueType outlines) are exactly what a marker needs, with real lowercase, curves and descenders.
- **Writing plane.** The plane through a taught TCP pose (default: the current tool-tip pose). Plane X is right, Y is up on the paper, and the marker points into the paper along +tool Z (pen-up is −tool Z). The marker is held perpendicular to the plane (tool Z = plane inward normal), re-orthonormalised from the surface normal so it is truly square; orientation is fixed per stroke, so the Cartesian moves are pure translation (well-conditioned, no wrist spin). An optional `tilt_deg` leans the pen.
- **Smooth strokes.** Each stroke is curvature-simplified (Ramer-Douglas-Peucker) to fit the 16-waypoint shared-memory limit and drawn as one blended MoveP at constant speed with no mid-stroke stops.
- **Consistent contact.** A small press-in (`press_mm`, default 1.5 mm) keeps the tip in contact despite minor height error; pair it with a spring-loaded pen holder on the real arm. Contact detection is automatically paused for the duration of a write (the marker pressing on paper is an expected contact) and restored afterward.

11.2 Use

Configure the marker as the TCP, jog the tip to the bottom-left corner of the paper held perpendicular (or select a saved writing plane), open the Write panel, type the text, set letter height (mm), pen-up clearance (mm) and draw speed (mm/s). Preview draws the strokes in the 3-D view and reports size and stroke / point counts (rejecting unreachable points); Write runs it (drives enabled); Stop aborts. Validation: HELLO 12 on a table plane → 11 strokes / 39 points, all IK-reachable at the marker TCP; a taught 200×140 mm plane "deskA" laid out "INTEGRA" fully reachable.

12. Writing Planes and Automatic Table-Plane Detection

12.1 Named writing planes

Instead of "the current pose", you can define, save and select named writing planes (bridge/planes.py, run/planes.json; the active plane is remembered). A **3-point teach** jogs the marker tip to the paper's bottom-left, bottom-right (+X) and top-left (+Y) corners, capturing each, and solves the plane frame (origin + right / up / into-paper axes) and its size. A quick "Save pose" stores the current tool-tip pose as a plane. Planes can be selected (made active), previewed in 3-D (rectangle + X/Y axes + strokes) and deleted. Storing explicit axes (xdir, ydir, zdir) rather than one quaternion avoids the left-handed reflection that would mirror text on a flat sheet.

12.2 Auto-detect the table by touch probing

The cobot finds the writing surface by itself, fully automated from the joints-0 home: it goes home, moves to a fixed joint configuration that holds the marker perpendicular to the base (pen straight down) over the front of the robot, descends straight down until it touches the table (sensed by the contact detector), and builds and activates a level writing plane at that height — no manual teaching. This is the standard "move until contact" touch-off used by UR (Polyscope "Until Contact"), CNC Z-probing and robot workpiece calibration.

Sequence: MoveJ home → MoveJ to the fixed ready joint config [-0.076, -0.442, -2.448, -0.435, 1.571, -1.495] (tool Z = base -Z) → arm the contact detector at high sensitivity and let it learn its baseline → coarse descend (guarded MoveL) until the detector trips (record tip z) → retract a few mm and re-arm → fine descend very slowly to a precise tip z → lift to the safe height → build a level plane at the touched point and save + activate it.

The ready pose is commanded in **joint space**, not as a Cartesian orientation target. Routing the pen-down orientation through the wrist-joint conventions came out inverted on the real arm; the fixed joint configuration is the exact angles the calibrated model shows as pen-straight-down, so it can never flip. Because the model matches the physical arm (joint-sign calibration), those joints make the real marker hold perpendicular.

Probing drives the tip toward the table on purpose: it approaches slowly in two stages (coarse then fine) so the marker is not damaged, uses the software contact detector to stop, and — crucially — relies on the independent hardware E-STOP / STO as the real guarantee. Use a spring-loaded marker holder and keep the workspace clear. In simulation a virtual table (EROBO_SIM_TABLE_Z=<z>) lets the whole probe run end to end (validated: touches land about 1 mm past the surface). Command: probe_plane with plane size, max drop, coarse / fine speeds, retract, sensitivity and an optional ready_joints override.

13. Contact Detection

On physical contact the arm applies a protective stop (drives disabled / hold) with an adjustable sensitivity — the "the robot touched something → stop" behaviour that industrial cobots implement without any added force/torque sensor.

13.1 Method

There is no strain gauge on this arm, so contact is inferred from the motor torque the drives already measure — the standard sensorless approach. Universal Robots estimate external effort from motor currents compared against the dynamic model and trip a protective stop past a user-set limit; the De Luca / DLR generalized-momentum residual observer estimates external joint torque without measuring acceleration. Three requirements those sources emphasise, all implemented here: a friction term (Coulomb + viscous), filtering (motor torque is noisy), and a velocity-dependent threshold (model error grows with speed).

core/include/erm/contact.hpp (ContactDetector):

```
residual_i      =  $\tau_{\text{measured}_i} - (\tau_{\text{model}_i} + \tau_{\text{friction}_i})$       # external-torque estimate
 $\tau_{\text{model}}$       = RNEA inverse dynamics + payload                      # erobot10 dynamics
 $\tau_{\text{friction}}$    =  $\text{Coulomb} \cdot \text{sign}(\dot{q}) + \text{viscous} \cdot \dot{q}$       # per-joint, joint-size scaled
residual_filtered += 0.20 * (residual - residual_filtered)      # low-pass EMA
threshold_i( $\dot{q}$ ) =  $\text{base}_i \cdot (0.15 + (1 - \text{sensitivity}) \cdot 1.85) + \text{kv}_i \cdot |\dot{q}_i|$  # sensitivity + speed term
contact when |residual_filtered_i| ≥ threshold_i for `debounce` (4) consecutive cycles
```

Per-joint base thresholds and speed slopes are scaled to actuator size. Sensitivity 0..1 maps the threshold from $\times 2.0$ (rugged, fewer false trips) to $\times 0.15$ (trips on a light touch) — the UR-style adjustable limit. The detector returns the per-joint residual (published as external torque), the worst joint and a margin. On real hardware the measured torque comes from the 0x6077 actual-torque PDO; in simulation it is the model + friction (+ an optional injected test torque, so it can be exercised offline). On contact both control nodes disable the drives, set `contact = 1` and publish the joint; re-enable (or freedrive) clears the stop, and it trips again if you are still pushing.

Contact detection is **OFF by default** and opt-in. On an uncalibrated arm the dynamic model does not match the drives well enough to distinguish a light touch from model error, so leaving it on nuisance-stops the drives; turn it on only after tuning friction and thresholds. It is a software convenience/quality layer on top of the independent hardware E-STOP / STO, never a substitute.

14. Safety System and Self-Collision

14.1 Three layers

- › **Hardware safety chain** — the E-STOP / STO circuit cuts drive power independently of all software. It is the only layer rated for personnel protection and must be wired and validated at installation.
- › **Real-time software monitor** (`safety.hpp`, control node, 1 kHz, on measured state): TCP / elbow overspeed, joint overspeed, workspace / plane penetration (with 5 mm / 15 % hysteresis), external-force limit, contact detection, and self-collision. Any trip latches a protective stop (drives disabled, brakes engaged, cause published and logged).
- › **Planner-side limiting** (before motion): the global speed override, reduced-zone scaling, deceleration into forbidden boundaries, torque-feasibility time-scaling, singular-path rejection and target validation. The planner keeps the robot away from the limits so layer 2 rarely has to act.

Additional guarantees: every trajectory is jerk-limited and C^2 -continuous; the control node hard-clamps the per-cycle setpoint step ($2\times$ rated speed) so no upstream fault can command a jump; e-stop / reset travel as edge counters that cannot be lost or replayed; a failed shared-memory read never moves the setpoint.

14.2 Self-collision

A capsule model of the links (`collision.hpp`, `erobo10_collision_geom.hpp`) predicts the minimum clearance between non-adjacent link pairs. The monitor publishes the clearance, the colliding capsule pair, a warning flag, and — just before modelled contact — auto-disables the drives (latched until re-enable). It is off by default (opt-in) to avoid false drive-offs on an uncalibrated arm, and is independent of the external-contact monitor; both feed the same protective-stop path.

14.3 Configuration and recovery

Safety configuration (in the controller / Studio) covers hard and reduced speed limits, TCP force limit, elbow speed, max reach, floor, ceiling, base keep-out, and up to six safety planes (forbidden or reduced). Changes apply live, are persisted (`safety.json`), logged with the acting user, and included in backups; collaborative defaults follow an ISO/TS 15066 flavour (250 mm/s reduced). Recovery: for a hardware or software e-stop, clear the hazard then Reset → Enable; for a protective stop the banner names the cause (overspeed, plane, contact, collision pair) — remove it and Enable; transient drive faults auto-recover; a persistent fault (more than four in fifteen seconds) latches and needs the cause fixed. Re-enabling never resumes stale motion: pending trajectories are cancelled on any drive drop-out and the setpoint re-synchronises to the measured position.

Residual-risk notice: the software monitors (collision, planes, force limits) are early-warning and mitigation features on standard hardware. They are **not** a certified safety function per ISO 13849 / IEC 62061 and do not replace a cell-level risk assessment, guarding, or the hardware E-STOP chain (ISO 10218-2 / ISO/TS 15066).

15. Joint Calibration

The 3-D model poses the arm from the encoder angles using `model_angle = sign·encoder + offset` per joint. If a joint turns the opposite way in the model, its sign is wrong; if the model's zero pose differs from the arm's, its offset is wrong. Both live in `run/calibration.txt` and are applied on the next control-node start — no rebuild:

```
sign    1  1  1  1  1  1    # 1 or -1 per joint J1..J6
offset  0  0  0  0  0  0    # DEGREES per joint J1..J6
```

Procedure (about five minutes, on the real arm, hand on the E-STOP):

- › **Signs.** Enable, send all joints to zero, then for each joint jog it about $+30^\circ$ and compare the model to the real arm; if it turns the opposite way, set that joint's sign to -1 . Put the six values on the sign line and restart the control node.
- › **Offsets.** With signs correct, send all joints to zero (the model shows the straight-up "candle"). If the real arm is not in that pose, for each joint line the arm up to a pose you can judge by eye, read the model's angle, and set `offset = true - shown (degrees)`. Restart.
- › **Verify.** All-zeros $\rightarrow$ model and arm both straight up; sweep each joint $\rightarrow$ the model tracks 1:1; a couple of Cartesian moves land where the model shows them.

Sign / offset map encoder $\leftrightarrow$ model consistently for both display and control, so once the model matches the arm, Cartesian (IK) moves are correct too. The control node prints the loaded calibration at startup. A geometrically wrong joint (elbow bends the wrong way even with sign / offset right) is a URDF axis / origin issue, not calibration. The ± 1 signs can also be set live from the interface (`set_sign`), which drops the drives to a safe hold for re-enable.

16. The Bridge Layer and Persistent Stores

The bridge translates JSON to and from shared memory. There are three surfaces, all speaking the same command set and attaching to the same block:

- `ws_bridge.py` — the WebSocket server (`ws://host:8765`) used by the desktop viewers. It maps JSON commands to shared memory, broadcasts state at about 50 Hz, and hosts the persistent stores and the application runners (writing, probing, palletizing, program execution). Its command handler is split into a v2 handler (programs, tools, safety, I/O, speed) and the core handler (motion, waypoints, TCP, application layer).
- `cobot_server.py` — a one-port HTTP server (`:8080`) that serves the UI and the control API (`GET /api/state`, `POST /api/cmd`) on the same origin, so a phone needs no second connection; `stdlib-only`.
- `shm_layout.py` — the ctypes mirror of the C++ layout, with `ShmClient` exposing `read_state`, `send_command`, `send_path`, `send_circular`, `set_payload`, `set_tcp`, `set_contact`, and the size asserts.

Persistent stores (all under `run/`, re-read on each operation so every UI stays in sync):

Store	File	Contents
WaypointStore (<code>waypoints.py</code>)	<code>waypoints.json</code> + <code>waypoints/<name>.json</code>	named joint-space waypoints
TcpStore / ToolLibrary (<code>tcp_config.py</code>)	<code>tcp_config.json</code> , <code>tools.json</code> + <code>tcp_configs/<name>.json</code>	active TCP + named tool library
plane store (<code>planes.py</code>)	<code>planes.json</code>	named writing planes + active plane
program store (<code>programs.py</code>)	via controller DB / JSON	typed-step programs
users (<code>users.py</code>)	<code>users.json</code> / <code>erobo.db</code>	users, roles, sessions
event log (<code>eventlog.py</code>)	<code>erobo.db</code> (SQLite)	event / alarm flight recorder

17. The Production Controller (erobo_controld)

controller/erobo_controld.py is the production controller service — one daemon owning every non-real-time controller duty while the real-time motion stays in the C++ control plane over shared memory. It provides:

- **WebSocket API** (:8765) — the live state stream plus the full command set (pendant / Studio).
- **REST API + UI** (:8080) — /api/v1/* for integrations, serves the Studio web build, plus the legacy /api/state and /api/cmd for the phone UI.
- **Modbus TCP slave** (:1502) — a PLC / SCADA register map (bridge/modbus_server.py).
- **MQTT publisher** (opt-in) — retained state and events to a broker.
- **Program store and typed-step executor** — motion / I/O / tool / speed / payload steps, loops and cycle counters.
- **Tool library** with active-TCP switching, **users / roles / sessions, settings**, a **SQLite event log / flight recorder**, usage counters and alarm summaries, and **backup / restore**.

Reliability work in this service fixed the drive-dropout root causes: CiA-402 fault reset now generates a true rising edge; transient drive faults auto-recover with a strike budget; EtherCAT link supervision performs staged automatic re-initialisation; and the published enabled reflects functional availability so the planner always cancels motion during recovery and never replays stale trajectories. The launch script scripts/run_hardware.sh starts the C++ control plane (control node, planner, IK solver) and then this controller.

18. API Reference

Three integration surfaces share one controller. All examples use the robot's IP. WebSocket and REST use JSON; add an id (or rid) to correlate replies.

18.1 WebSocket API (ws://robot:8765)

A {"type":"state"} frame is broadcast to every client at about 50 Hz, carrying joints, TCP and flange pose, mode, enabled / estop / moving / freedrive, progress, manipulability, safety fields, telemetry, and contact / collision fields. Replies are {"type":"ack",...}; there are also welcome, error, waypoints, tools, planes, program, writing, probe, signs, and ready_pose message types.

18.1.1 Motion and drive control

Command	Purpose
move_joint {q[6], speed, accel}	MoveJ in joint space
move_ptp {pos[3], rpy[3] or quat[4], speed}	MoveJ to a Cartesian pose (IK once)
move_lin {pos[3], rpy[3] or quat[4], speed}	MoveL straight line
move_p {waypoints[], blend, tool_speed, tool_acc}	MoveP constant-speed blended path
move_c {via, end, full}	MoveC arc / full circle
move_rel {delta[3], frame}	relative offset (base or tool frame)
jog {frame joint/base/tool, vel[6], speed}	velocity jog (expires after 250 ms — resend while held)
stop / hold	smooth decel to rest / hold current setpoint
enable / disable	power-enable / disable the drives
estop / reset	protective stop / clear it
freedrive {on}	hand-guiding (brakes off, zero-torque)
ecat_reset	restart the EtherCAT connection (clear faults, recover dropped joints)

18.1.2 Programs, waypoints and application layer

Command	Purpose
program_run {steps[], speed, accel, loop}	run a waypoint program
program_pause / program_resume	feed-hold pause / resume on the path
program_stop	stop the running program / writing / probe
prog_list / prog_get / prog_save / prog_delete / prog_run	typed-step program store (controller)
wp_list / wp_save / wp_update / wp_delete / wp_goto	named waypoints (save / edit / rename / delete / go)
write_preview / write_run {text, height, pen_up, draw_speed, tilt_deg, press_mm}	robot writing
plane_save / plane_list / plane_select / plane_delete	writing planes
probe_plane {name, width, height, max_drop, coarse_speed, fine_speed, sensitivity, ready_joints}	auto-detect the table by touch
pallet_run {pick, corner, nx, ny, nz, dx, dy, dz, approach, order}	palletizing
save_ready_pose / get_ready_pose	store / read the perpendicular ready pose

18.1.3 Configuration and tooling

Command	Purpose
set_speed {frac}	global speed override (0..1)
set_payload {kg, cog}	tool / payload mass and CoG
set_tcp {pos, quat/rpy/rotpvec, payload, cog} / get_tcp	set / read the active TCP
tcp_teach {flanges[], apply}	4-point TCP position teach
tcp_teach_orient {ref, zdir, xdir, apply}	orientation teach (point-along-axes)
tool_list / tool_save / tool_update / tool_rename / tool_select / tool_delete	the tool library (list / create / edit / rename / apply / delete)
set_contact {on, sensitivity} / contact_poke {joint}	contact detector (poke is simulation only)
set_collision {on}	self-collision monitor
set_sign {signs[6]} / get_signs	live per-joint direction sign
get_safety / set_safety {config}	safety configuration
io_get / `io_set {do, value	ao, volts} / di_force {di, value}` digital / analog I/O
get_state	one-shot state

18.2 REST API (<http://robot:8080/api/v1>)

Auth: POST /auth/login {user, pin} returns {token}; send Authorization: Bearer <token>. Reads are open; writes need operator or higher; safety / users / settings / backup need admin. Reads: GET /state, /system, /metrics, /events?since&level&limit, /programs[/id], /waypoints, /tools, /safety, /io, /settings, /backup. Writes: POST /programs, /programs/{id}/run, /run/stop, /command {any WS command}, /safety, /io, /settings, /restore {data: base64 tar.gz}, /users; PUT /programs/{id}; DELETE /programs/{id}, /users/{name}, /tools/{name}. The legacy phone endpoints /api/state and /api/cmd remain.

18.3 Modbus TCP (robot:1502, unit id 1)

Area	Address	Meaning
Coils (FC1/5/15)	0-15	digital outputs DO0-15
Discrete in (FC2)	0-15	digital inputs DI0-15
Input regs (FC4)	0-12	mode, enabled, e-stop, moving, progress%, speed %, safety zone, safety code, stop-bits, prog running, prog cycles, uptime s, jitter μ s
Input regs (FC4)	16-27	q1-6 [mrad], qd1-6 [mrad/s]
Input regs (FC4)	28-33	TCP x,y,z [0.1 mm], roll,pitch,yaw [mrad]
Input regs (FC4)	34-47	τ 1-6 [0.1 N·m], TCP speed [mm/s], ext force [0.1 N], temp1-6 [0.1 °C]
Holding (FC3/6/16)	0	speed override $\times 1000$ (50-1000)
Holding (FC3/6/16)	1	command: 1 start prog(reg 2), 2 stop, 3 enable, 4 disable, 5 reset, 6 E-STOP, 7/8 freedrive on/off
Holding (FC3/6/16)	2	program number (1-based, creation order)
Holding (FC3/6/16)	3	last command result (1 ok / 2 error)

18.4 MQTT (opt-in)

Publishes retained JSON state to <prefix>/state at the configured interval and events to <prefix>/events (MQTT 3.1.1, QoS 0). Vision systems integrate through the REST / WS motion API — move_rel with a measured offset is the standard visual-servo primitive, and probe_plane / plane_* provide taught work-plane management.

19. User Interfaces

19.1 erobo10_viewer.html — desktop 3-D viewer

A Three.js desktop viewer and control panel that connects over WebSocket. It loads the real STL meshes (falling back to embedded decimated meshes or primitive cylinders if the STLs are not reachable) and computes the same URDF forward kinematics in JavaScript to place each link, so the model is a live digital twin. Panels: joint jog and Cartesian move, drag-teach, the named-waypoint panel (with Go / Edit / Rename / Save as / Delete), the TCP panel (numeric editor, 4-point and orientation teach, and the saved tool library with Apply / Edit / Rename / Save as / Delete), the Contact panel, the Write panel with writing planes and auto-detect, the Motion+ panel (palletizing and arc), live telemetry, the programmed-motion panel with the single Run / Pause / Resume button, and the ± 1 joint-sign toggles. Safety events (collision, contact, singularity, drive-fault, name-exists) appear as corner toasts; a drive-fault toast prompts the Restart EtherCAT button when a joint drive drops out unexpectedly.

19.2 cobot_mobile.html — phone UI

A landscape touch UI served same-origin over HTTP (:8080/): translate / rotate joysticks, per-joint D-pads, a mode toggle, teach, payload, view presets, the named-waypoint panel and a 3-D view. It needs no second connection because the control API is on the same origin.

19.3 eRoBo Studio — Electron + React pendant

studio/ is an Electron + React + TypeScript application; the same build runs three ways — a packaged desktop app (Windows NSIS, Linux AppImage/deb, macOS dmg), the on-robot web UI served from studio/dist/ at :8080/, and a Vite dev server. It is a pure network client of the controller (WebSocket :8765, REST :8080) pointed at any robot from Settings. Features: a live 3-D digital twin (real meshes, TCP trace, safety-plane rendering, a ghost preview rig), jog + hand-guiding + move-to, a full program editor with live step tracking, an I/O panel, safety configuration, a tool / TCP teach wizard, per-joint health dashboards with sparklines, a system-log viewer, and dark / light themes. The source is organized into pages (Move, Program, Io, Safety, Tools, Diag, Logs, Settings), components, a Zustand store, an API client, and a Three.js viewport with its own kinematics.

20. Hardware Reference — ZeroErr eRob Actuators

Every joint is a ZeroErr **eRob** all-in-one rotary actuator: servo driver + dual absolute encoders + frameless torque motor + armature-actuated friction brake + strain-wave gear, in one housing. Positive rotation is CCW facing the output shaft; hardware direction is fixed, only the command sign changes it.

20.1 Joint to actuator map

Joint	Actuator	Gear ratio	Role
J1	eRob 142H	120	base / shoulder yaw
J2	eRob 142H	120	shoulder pitch
J3	eRob 110H	120	elbow
J4	eRob 80H	100	wrist 1
J5	eRob 80H	100	wrist 2
J6	eRob 80H	100	wrist 3 / flange

20.2 Torque, speed, power, encoders

- **Torque ceilings** (avg-load-max for the fitted gear ratios) used by the software:
JOINT_EFFORT = {281, 281, 140, 51, 51, 51} N·m for J1..J6. Four tiers exist per actuator (rated continuous, avg-load-max, peak start/stop, momentary max — never exceed the last).
- **Velocity ceilings** $\approx$ 90 % of actuator max output speed: {1.57, 1.57, 2.35, 2.80, 2.80, 2.80} rad/s, with a hard counts/s clamp {145927, 145927, 218453, 262144, 262144, 262144}.
- **Power**: nominal 48 V DC $\pm$ 10 % (bus OK 44–55 V; absolute max 60 V). Rated current at 48 V: 80H 3.4 A, 110H 18.6 A, 142H 26 A $\rightarrow$ arm aggregate $\approx$ 80.8 A ($\sim$ 3.9 kW) rated, peak higher — size the PSU and wiring accordingly. Hard deceleration on the heavy 142H joints can push the bus into over-voltage (regen); the jerk-limited planner mitigates this, and a bleeder / super-cap helps for aggressive moves.
- **Encoders**: motor 17-bit, output 19-bit single-turn = 524288 counts/rev; position object 0x6064 with a default zero at 262144. Software position limits via 0x607D.

20.3 Brake, STO and freedrive

Enable releases the brake; disable engages it (actuation about 150 ms). After enabling, wait at least 500 ms before motion, and keep enable $\leftrightarrow$ disable at least 300 ms apart. A fault / e-stop at high speed and heavy load can slam the brake and irreversibly damage the gear — the brake is not a high-speed stop; the correct emergency stop is the hardware **STO** (Safe Torque Off), which cuts motor power independently while the drive stays powered. Freedrive uses the manual's recommended back-driving method: CST torque mode with target torque zero and the drive enabled, which needs less force than releasing the brake alone; a future smoothness upgrade would push RNEA gravity torque into the torque feed-forward (0x60B2), which the dynamics model already computes.

20.4 CiA-402 objects used

Object	Dir	Meaning
0x6040 / 0x6041	Rx / Tx	controlword / statusword (state machine)
0x6060 / 0x6061	Rx / Tx	modes of operation: 8 CSP, 9 CSV, 10 CST (freedrive)
0x6064 / 0x606C	Tx	actual position / velocity (counts)
0x607A / 0x60FF	Rx	target position (CSP) / target velocity (CSV)
0x6071 / 0x6077	Rx / Tx	target torque (CST) / actual torque
0x6078 / 0x6079 / 0x22A2	Tx	motor current / DC-link voltage / temperature
0x4602 / 0x60FE	SDO / Rx	brake release / digital outputs
0x603F	Tx	error code

21. EtherCAT Integration

The upper stack (UI → bridge → IK → planner → control loop) is identical in simulation and on hardware; only the bottom layer switches from the sim model to real drives. Slave identity: vendor 0x5a65726f, product 0x00029252. The drives run **CSP** (Cyclic-Sync Position) in the current hardware node — the planner's jerk-limited position setpoint is converted to encoder counts and written to 0x607A, and the drive closes the position loop and stops exactly. (An earlier variant used CSV velocity mode with a software position loop $vel = qd_ff + KP \cdot (q_cmd - q_act)$; the two are interchangeable by changing the mapped object and mode.) Distributed Clocks (1 ms SYNC0) are on by default for tight sync.

Bring-up sequence: build the hardware node (make `HARDWARE=1` apps, links `-lethercat`), bring up the IgH master, then use `--selftest` to verify all six drives reach Operation Enabled and print their state and encoder readings with nothing moving; `--selftest --allow-motion` adds tiny clamped per-joint nudges; `--set-home <q0..q5>` captures the current encoders as a reference pose and writes the offsets. Calibrate in order — encoder scale, direction sign, home offset, brake polarity, gain, then raise speeds — via `run/calibration.txt` (and, for the CSV variant, `config/ecat_calib.conf`) with no rebuild. First power-on: low speed caps, one joint at a time, hand on the hardware E-STOP; the software refuses all motion until you explicitly enable, and there is no jump on enable because the planner tracks the measured position while disabled. Run the loop with real-time priority (`--rt / SCHED_FIFO + mlockall`) and DC sync for jitter-free 1 kHz.

22. Build System

The build is plain `make` + `g++` (C++17), with no external C++ libraries (the engine is header-only), so it ports to a real-time control box. Each `apps/*.cpp` and `test/*.cpp` is a small `main()` that includes the `erm::` headers and compiles into `bin/`.

Target	Result
<code>make</code>	simulation apps (<code>control_node</code> , <code>motion_planner</code> , <code>ik_solver</code>) + tools + unit tests
<code>make HARDWARE=1 apps</code>	additionally builds <code>ecat_control_node</code> (links <code>-lethercat</code>)
<code>make test</code>	builds and runs the unit tests (<code>test_fk</code> , <code>test_ik</code> , <code>test_traj</code> , <code>test_safety</code>)
<code>make test-ecat-mock</code>	compiles / links the EtherCAT node against a stand-in <code>ecrt.h</code> (no hardware, no <code>libethercat</code>)
<code>make clean</code>	removes <code>bin/</code>

The Python side needs only `websockets` (for `ws_bridge.py`; `cobot_server.py` is `stdlib`-only) and `NumPy` for the test oracle. The Python shared-memory mirror is validated at import by size asserts, so a layout drift between C++ and Python is caught immediately. eRoBo Studio is built separately with `npm install && npm run build` (or `npm run package` for installers).

23. Deployment

deploy/ contains the production deployment assets:

- `install.sh` — installs the platform and enables the `systemd` services.
- `systemd/*.service` — RT-scheduled units: `erobo-control`, `erobo-planner`, `erobo-ik`, `erobo-controller`, and `erobo-sim` (the simulator).
- `update.sh` — a safe updater with automatic backup.
- `Dockerfile.sim` + `docker-entry.sh` — a Docker simulator image (`docker build -f deploy/Dockerfile.sim -t erobo-sim . && docker run -p 8080:8080 -p 8765:8765 -p 1502:1502 erobo-sim`).

24. Running the Stack

24.1 Simulation (no hardware)

```
make && make test          # build + unit suites
./scripts/run.sh           # sim control plane: control_node + planner + ik_solver + bridge
# open http://localhost:8080 → Enable → jog / teach / program / run
python3 test/integration_test.py  # 40+ end-to-end acceptance checks
```

24.2 Real robot

```
make HARDWARE=1 apps      # needs the IgH EtherCAT master
bash scripts/run_hardware.sh
# phone (landscape): http://<robot-ip>:8080/
# desktop viewer:    http://<robot-ip>:8080/tools/erobo10_viewer.html
```

scripts/run_hardware.sh removes any stale shared block, starts ecat_control_node as root (Distributed Clocks on unless EROBO_NODC=1), waits for the block to appear and makes it world-writable, starts motion_planner and ik_solver, and then execs the controller (controller/erobo_control.py), which serves the WebSocket, REST and Modbus surfaces and the Studio UI. Other helper scripts: scripts/build.sh (compile + test), scripts/serve_viewer.sh (serve the project over HTTP so the desktop viewer can load the STL meshes).

After changing any C++ file (for example the planner feed-hold), rebuild the affected node (make HARDWARE=1 apps) and restart the stack for the change to take effect. Python and HTML changes (the bridge and the viewer) need only a bridge restart and a browser refresh.

25. Testing and Verification

Two layers of testing:

- › **C++ unit tests** (deterministic, no hardware): `test_fk` (FK orthonormality + Jacobian vs finite difference + DH-FK vs the NumPy URDF oracle to $1.7\text{e-}6$ m), `test_ik` (3000 random poses, 100 % convergence, sub- μm , smooth continuity), `test_traj` (velocity / accel / jerk within limits, synchronized arrival, straight-line deviation sub- μm), `test_safety`, `test_tcp` (solveKtcp round-trip $1.1\text{e-}8$ m, MoveL tip-straight / flange-curved, payload $\rightarrow$ torque), plus `test_csp_mock` and `test_motion`.
- › **Integration** (all processes running): `test/integration_test.py` drives real moves through the full command $\rightarrow$ IK $\rightarrow$ planner $\rightarrow$ control $\rightarrow$ state loop and asserts the robot reaches the target smoothly, plus the controller surfaces (state machine, override, zones, I/O, programs, auth, Modbus, events, backup). The NumPy oracle (`urdf_fk_oracle.py`) is the independent ground truth that proves the C++ kinematics equal the URDF.

The EtherCAT node can be exercised offline against two mocks: `test/mock_ecrt_sim/ecrt.h` is a functional drive simulator (CiA-402 state machine + velocity integration + modes-of-operation / SDO) so the node can be run without hardware, and `test/mock_ecrt/ecrt.h` is a minimal API surface for pure compile-testing.

26. Recent Feature Additions

This section records the most recent engineering changes, all validated in isolation before delivery.

26.1 TCP application to programmed motion

Ensured the active TCP governs stored-waypoint and programmed motion, not just directly-commanded Cartesian moves. The IK solver and planner already applied the tool for PTP / LIN / P / C; the gap was that a stored waypoint replayed raw joint angles (TCP-independent). Now, when a tool is applied and a waypoint carries a taught pose, the step is replayed as PTP to the tool-tip pose, so Apply TCP visibly changes where joint / PTP / linear / arc programmed motion lands, while the no-tool case keeps exact joint replay.

26.2 The named tool library

A multi-tool library layered on the existing single-active-TCP: save, apply (select), edit in place, rename, save-as (copy) and delete, exposed in the viewer's TCP panel. The backend (ToolLibrary + tool_list / tool_save / tool_update / tool_rename / tool_select / tool_delete) keeps tools.json and syncs the active TCP when the active tool is renamed.

26.3 Drive-fault popup

A sticky corner popup that appears when a joint drive drops from operable to inoperable unexpectedly (the per-joint Operation-Enabled bit flips off while the drives were enabled and the user did not command a disable / e-stop / sign change / contact stop). It tells the operator to press Restart EtherCAT and offers the action inline, clearing on recovery.

26.4 On-path feed-hold pause

Program play / pause that stops the arm exactly on its path and resumes the same motion, via runtime velocity scaling in the planner (CMD_PAUSE / CMD_RESUME ramp a pause_scale to zero and back). The interface is a single Run / Pause / Resume button; the bridge freezes the step-completion timeout while held.

26.5 Per-item files

Every saved waypoint and tool is also written as an individual JSON file — run/waypoints/<name>.json and run/tcp_configs/<name>.json — created on save, moved on rename, copied on save-as, and removed on delete, while the aggregate waypoints.json / tools.json are kept intact.

26.6 Unique-name enforcement and item editing

Saving a waypoint or tool with a name that already exists is rejected (case-insensitive) with a clear message, enforced both client-side and in the backend. Both lists gained Edit (change values), Rename (change name, moving the per-item file), and Save As (copy under a new name).

27. Complete File and Directory Reference

This chapter lists every folder and file in the project with a one-line description, grouped by directory, for quick navigation. Build output (bin/), Python caches (__pycache__/), runtime state (run/), and personal/confidential files are git-ignored and are noted where relevant.

27.1 Top-level layout

Path	What it is
apps/	The C++ real-time control-plane executables (sim node, EtherCAT driver, planner, IK solver)
core/include/erm/	Header-only, dependency-free motion library (math, kinematics, dynamics, IK, trajectories, TCP, contact, collision, safety, shared-memory contract)
core/	The robot model consumed by the code generators
bridge/	Python JSON to shared-memory translation layer, application runners, and persistent stores
controller/	The production controller daemon (WebSocket / REST / Modbus / MQTT, programs, users, events)
config/	EtherCAT runtime calibration template
robot/	URDF robot description + STL visual meshes
tools/	Operator/developer front-ends (3-D viewers, phone UI) and CLI/GUI test drivers
scripts/	One-command build and launch helpers
deploy/	Installer, systemd services, Docker image, updater
test/	Unit and integration test suites + EtherCAT mocks
studio/	eRoBo Studio — the Electron + React + TypeScript pendant application
docs/	The documentation set (including this reference)
run/	Runtime state written at run time (git-ignored)
bin/	Compiled binaries (git-ignored build output)

27.2 Root files

File	Description
README.md	Project entry point: overview, quick start (sim + hardware), repository map, engineering notes
RELEASE_NOTES.md	Platform 2.0 release notes — safety system, controller, Studio, deployment, reliability fixes, verification
Makefile	Build system: make (sim apps + tools + tests), make HARDWARE=1 apps (EtherCAT node), make test, make test-ecat-mock, make clean
.gitignore	Ignores build output, runtime state, dependencies, and personal/confidential files
.gitattributes	Git attributes (line-ending / language settings)
font_preview_hershey.png	Rendered preview of the Hershey single-stroke writing font

27.3 apps/ — real-time control plane

File	Description
ecat_control_node.cpp	THE hardware driver: brings up the IgH EtherCAT master and the CiA-402 drive state machine for six servo drives, streams 1 kHz CSP position setpoints, runs freedrive (CST torque mode), publishes telemetry and TCP pose, loads run/calibration.txt, and runs the collision/contact safety hooks
control_node.cpp	The simulation twin of the EtherCAT node — same shared-memory role but an internal SimDrives model; supports a virtual table via EROBO_SIM_TABLE_Z for offline touch-probe testing
motion_planner.cpp	1 kHz jerk-limited (S-curve) trajectory generator: Move/L/P/C, jog, stop, global speed override, safety-zone scaling, torque-feasible de-rate, singularity handling, and the on-path feed-hold pause
ik_solver.cpp	Turns UI goals into joint targets: joint passthrough, PTP (IK once), LIN/P/C reachability validation; applies the active TCP and the torque-feasibility gate

27.4 core/include/erm/ — header-only motion library

File	Description
linalg.hpp	SE(3)/SO(3) math: Vec3/Mat3/Mat4, quaternions, rotation vectors, SLERP, SO(3) log/exp, dense linear solver
kinematics.hpp	Forward kinematics and the geometric Jacobian over the DH model; Yoshikawa manipulability
erobo10_dh.hpp	Auto-generated, URDF-validated Denavit-Hartenberg table + per-joint limits (position, velocity, accel, jerk, effort)
erobo10_dyn.hpp	Auto-generated per-joint frames + link rigid-body inertials (mass, centre of mass, inertia)
dynamics.hpp	Recursive Newton-Euler inverse dynamics ($\tau = M \ddot{q} + C \dot{q} + G$) with an optional flange payload
ik.hpp	Robust 6-DOF inverse kinematics: Levenberg-Marquardt damped least squares, quaternion error, joint-limit clamping, warm start, multi-seed restarts
trajectory.hpp	Jerk-limited S-curve profiles; JointTrajectory (PTP), CartesianTrajectory (LIN), BlendedPath (MoveP), CircularTrajectory (MoveC)
tcp.hpp	The Tool-Center-Point layer: tcpFK, solveIKtcp, and tool transforms from pos+quat / rpy / rotnvec
contact.hpp	Sensorless external-torque contact detector (generalized-momentum residual + friction + velocity-dependent threshold)
collision.hpp	Capsule self-collision distance model and the clearance monitor
erobo10_collision_geom.hpp	The per-link capsule collision geometry for the arm
safety.hpp	The measured-state safety monitor: speed / force / reach / floor / ceiling / planes / zones
shm.hpp	POSIX shared-memory map/unmap helpers + the lock-free seqlock read/write primitives
shm_layout.hpp	The canonical shared-memory contract: the five regions, the CmdType enum, struct sizes, magic and version
hal.hpp	EtherCAT hardware-abstraction shape (array types, cyclic interface) shared by the nodes
ecat_config.hpp	Tombstone header — the EtherCAT tunables now live at the top of apps/ecat_control_node.cpp
ecat_hal.hpp	Tombstone header — the EtherCAT HAL now lives inside apps/ecat_control_node.cpp

core/erobo10.urdf is the robot model the code generators consume; core/src/ is reserved (empty).

27.5 bridge/ — JSON to shared memory + persistent stores

File	Description
ws_bridge.py	WebSocket server (:8765): maps JSON commands to shared memory, broadcasts state at ~50 Hz, and hosts the application runners (writing, touch-probe, palletizing, program executor) and the stores
cobot_server.py	One-port HTTP server (:8080): serves the UI and the control API (/api/state, /api/cmd) on the same origin for the phone UI (standard library only)
shm_layout.py	ctypes mirror of shm_layout.hpp (byte-identical, size-asserted) + the ShmClient helper
waypoints.py	WaypointStore — named joint-space waypoints (waypoints.json + per-item files)
tcp_config.py	TcpStore + ToolLibrary — the active TCP, the named tool library, and the 4-point / orientation calibration solvers
planes.py	Writing-plane store + the 3-point plane solver
programs.py	Typed-step program store (motion / I/O / tool / speed / payload steps)
palletizing.py	Pallet pattern generator — a pick/place grid expanded into a completion-gated program
writing.py	Text to single-stroke (Hershey) toolpath generator with RDP simplification
users.py	Users, roles, and API sessions
eventlog.py	SQLite event log / alarm flight-recorder
modbus_server.py	Modbus TCP slave (the PLC/SCADA register map)
requirements.txt	Python dependencies (websockets, etc.)
fonts/futural.jhf	The Hershey single-stroke "futural" font used by the writing feature

27.6 controller/ — production controller

File	Description
erobo_controld.py	The production controller daemon: WebSocket (:8765) + REST/UI (:8080) + Modbus (:1502) + optional MQTT, the program store and typed-step executor, tool library, users/roles/sessions, settings, SQLite event log, metrics, backup/restore; serves the Studio web build

27.7 config/

File	Description
ecat_calib.conf	EtherCAT runtime calibration template — per-joint direction, offset, counts-per-rev, gain, velocity clamp, and brake polarity (auto-loaded, no rebuild)

27.8 robot/ — robot description and meshes

File	Description
erobo10.urdf	The robot model (links, joints, limits, visual mesh references)
erobo10_corrected.urdf	Corrected URDF variant
meshes/erobo10/base_link.STL	Base link visual mesh
meshes/erobo10/link1.STL ... link6.STL	Visual meshes for links 1 through 6 (rendered by the viewers)

27.9 tools/ — front-ends and utilities

File	Description
erobo10_viewer.html	The desktop 3-D viewer and control panel (Three.js): joint jog, drag-teach, the waypoint panel, the TCP panel + tool library, the contact panel, the writing panel with planes and auto-detect, palletizing, telemetry, the Run/Pause/Resume programmed-motion panel, and the ± 1 joint-sign toggles
cobot_mobile.html	The landscape phone touch UI (joysticks, D-pads, teach, payload, view presets, 3-D), served same-origin over HTTP
cobot_meshes.js	Decimated, embedded copies of the meshes so the viewers render the arm even without the full STLs
inject.cpp	CLI command injector straight into shared memory — a bridge stand-in for low-level testing
pose_cli.py	Terminal WebSocket client (ptp / lin / joint / monitor / freedrive / teach) with an offline FK helper
pose_tester.py	tkinter GUI to test MoveJ / MoveL / MoveP with a realistic waypoint randomizer
gen_meshes_from_stl.py	Decimate the STL meshes into the embedded <code>cobot_meshes.js</code>
gen_model_from_urdf.py	Generate the DH and dynamics headers from the URDF
meshes/erobo10/*.STL	STL mesh copies used by the tools

27.10 scripts/ — build and launch

File	Description
build.sh	Compile the project and run the unit tests
run.sh	Start the simulation control plane (<code>control_node</code> + <code>motion_planner</code> + <code>ik_solver</code> + <code>bridge</code>)
run_hardware.sh	Start the real stack (<code>ecat_control_node</code> as root + <code>planner</code> + <code>ik_solver</code> + the controller)
serve_viewer.sh	Serve the project over HTTP so the desktop viewer can load the STL meshes

27.11 deploy/ — installer, services, Docker

File	Description
install.sh	Install the platform and enable the systemd services
update.sh	Safe updater with automatic backup
Dockerfile.sim	Docker image that runs the full simulator
docker-entry.sh	Container entry point for the simulator image
systemd/erobo-control.service	RT-scheduled unit for the control node
systemd/erobo-planner.service	RT-scheduled unit for the motion planner
systemd/erobo-ik.service	Unit for the IK solver
systemd/erobo-controller.service	Unit for the production controller
systemd/erobo-sim.service	Unit for the simulator node

27.12 test/ — unit and integration tests

File	Description
test_fk.cpp	Forward-kinematics orthonormality + Jacobian vs finite-difference + DH-FK vs the NumPy URDF oracle
test_ik.cpp	IK round-trip: convergence, sub-micrometre precision, continuity over thousands of poses
test_traj.cpp	Jerk-limited profile limits, synchronized PTP arrival, straight-line LIN deviation
test_safety.cpp	Safety-monitor unit tests
test_tcp.cpp	solveKtcp round-trip, MoveL tip-straight vs flange-curved, payload-CoG to torque
test_csp_mock.cpp	The CSP servo loop validated against the drive-simulator mock
test_motion.cpp	Motion integration checks
test_ecat_compile.cpp	Tombstone (the obsolete HAL compile test)
integration_test.py	40+ full-stack acceptance checks (state machine, accuracy, override, zones, I/O, programs, auth, Modbus, events, backup)
urdf_fk_oracle.py	Independent NumPy URDF forward-kinematics oracle (the ground truth for the tests)
mock_ecrt_sim/ecrt.h	A functional EtherCAT drive simulator (CiA-402 state machine + velocity integration + SDO) so the driver can be run offline
mock_ecrt/ecrt.h	A minimal ecrt.h API surface for pure compile-testing

27.13 studio/ — eRoBo Studio (Electron + React + TypeScript)

File	Description
README.md	Studio build and run instructions (desktop app, on-robot web UI, dev server)
package.json	Node package manifest and scripts
vite.config.ts	Vite build configuration
tsconfig.json	TypeScript compiler configuration
index.html	Vite HTML entry point
electron/main.cjs	Electron main process (the desktop shell)
electron/preload.cjs	Electron preload bridge (secure IPC)
src/main.tsx	React application entry point
src/App.tsx	Root component and page routing
src/store.ts	Zustand global state store
src/theme.css	Theming (dark / light)
src/api/robot.ts	The WebSocket / REST API client
src/components/Header.tsx	App header
src/components/Nav.tsx	Navigation bar
src/components/StatusBar.tsx	Live status bar
src/components/Toasts.tsx	Notification toasts
src/components/ConnectOverlay.tsx	Connection overlay
src/components/LoginModal.tsx	Login modal
src/pages/MovePage.tsx	Jog / move-to / hand-guiding screen
src/pages/ProgramPage.tsx	Program editor with live step tracking
src/pages/IoPage.tsx	Digital / analog I/O panel
src/pages/SafetyPage.tsx	Safety configuration
src/pages/ToolsPage.tsx	Tool / TCP teach wizard
src/pages/DiagPage.tsx	Per-joint health dashboards
src/pages/LogsPage.tsx	System log viewer
src/pages/SettingsPage.tsx	Settings (robot address, theme, etc.)
src/three/Viewport3D.tsx	The live 3-D digital-twin viewport
src/three/kinematics.ts	JavaScript forward kinematics for the twin
dist/index.html + dist/assets/*	The committed production web build served on-robot at :8080

27.14 run/ — runtime state (git-ignored)

File	Description
calibration.txt	Per-joint direction sign and offset, loaded by the EtherCAT node on start
tcp_config.json	The currently active TCP
tcp_configs/	One JSON file per saved tool in the library
waypoints.json	The named-waypoint list
waypoints/	One JSON file per saved waypoint (e.g. home.json, pick.json, place.json)
users.json	Users, roles, and PINs
erobo.db	SQLite event log / flight recorder

27.15 docs/ — documentation set

File	Description
ARCHITECTURE.md, FLOW_AND_ARCHITECTURE.md	System architecture, the two planes, and the byte-by-byte data flow
DH_PARAMETERS.md, URDF_MIGRATION.md	The kinematic model and the URDF migration
MOTION_TYPES.md	Research-backed MoveJ / MoveL / MoveP / MoveC definitions
HARDWARE_ACTUATORS.md	The ZeroErr eRob actuator reference (torque/speed, power, encoders, brake, CiA-402 objects)
ETHERCAT_INTEGRATION.md, HARDWARE_BRINGUP.md, RUN_ON_HARDWARE.md, INTEGRATION.md	Hardware integration, bring-up, calibration, and running on the real arm
TCP_CONFIGURATION.md, TCP_RESEARCH_AND_DESIGN.md	TCP usage and the design rationale
CALIBRATION.md	Joint sign/offset calibration procedure
CONTACT_DETECTION.md, AUTO_PLANE.md, WRITING.md	The contact-detection, auto-plane-probe, and robot-writing features
SAFETY.md, FAT_SAT.md	The safety manual and factory/site acceptance tests
PROTOCOL.md, POSTMAN.md, INTEGRATION_API.md	The WebSocket/REST/Modbus/MQTT API surface
PHONE_UI.md, REMOTE_UI_SETUP.md, USER_MANUAL.md	The phone UI, remote-UI setup, and operator manual
INSTALLATION.md, RUN_AND_TEST_GUIDE.md, TROUBLESHOOTING.md	Installation, running/testing, and troubleshooting
erobo10_FULL_DOCUMENTATION.pdf	This consolidated technical reference

28. Troubleshooting

Symptom	Likely cause and fix
Drives never reach Operation Enabled	Wiring / limits / STO; verify vendor/product IDs and that all six slaves enumerate (ethercat slaves); watch the per-joint CiA-402 state in the node log
A joint runs away on enable (CSV)	Wrong direction sign (positive feedback) — flip that joint's sign in run/calibration.txt and restart
Model turns opposite to the arm / pose diverges with angle	Joint sign wrong — run the calibration sign procedure
Model shape shifted at all-zeros	Joint offsets wrong — run the offset procedure
Joint sags / won't hold	Brake polarity or position gain too low
Audible buzz / vibration	Position gain too high, or enable Distributed Clocks for tighter sync
Jitter	Enable DC (1 ms sync0) and run the loop with RT priority + CPU isolation
A drive drops out mid-run	The drive-fault popup appears — press Restart EtherCAT, then Enable; if it re-latches persistently, read the 0x603F error code in the log
MoveL stops short	The straight path is infeasible here (singularity / joint limit) — use MoveJ/PTP or split into shorter segments
Contact detector nuisance-trips	It is opt-in; tune friction and thresholds, lower the sensitivity, or leave it off (the hardware STO still protects the arm)
Pause finishes the current move instead of stopping immediately	The C++ planner needs rebuilding (make <code>HARDWARE=1</code> apps) to pick up the feed-hold; until then it falls back to holding at the next waypoint
Can't save a waypoint / tool	The name already exists (case-insensitive) — choose a different name, or use Save As

29. Safety and Residual-Risk Notice

The hardware E-STOP / STO circuit is the only layer rated for personnel protection and must be wired and validated at installation, independent of all software. The software features described here — contact detection, self-collision prediction, safety planes / zones, speed and force limits, the software e-stop — are early-warning and mitigation features implemented on standard hardware. They are **not** a certified safety function per ISO 13849 / IEC 62061 and do not replace a cell-level risk assessment, guarding, or the hardware safety chain. Perform a risk assessment (ISO 10218-2 / ISO/TS 15066) before collaborative operation. Support the arm before engaging freedrive (it can sag under gravity), keep the workspace clear and a hand on the E-STOP during any touch-probing or first power-on, and never rely on the friction brake as a high-speed stop.

30. Glossary

- **CiA-402** — the CANopen drive-and-motion profile (state machine, modes of operation) used over EtherCAT (CoE).
- **CSP / CSV / CST** — Cyclic-Sync Position / Velocity / Torque modes of operation (0x6060 = 8 / 9 / 10).
- **DC (Distributed Clocks)** — EtherCAT time synchronisation (1 ms SYNC0 here) for jitter-free cyclic exchange.
- **DH parameters** — Denavit-Hartenberg description of the kinematic chain.
- **Feed-hold** — pausing motion by scaling trajectory velocity to zero while staying on the path; resume ramps it back.
- **FK / IK** — forward / inverse kinematics.
- **Jerk-limited / S-curve** — a motion profile with bounded jerk (rate of change of acceleration) for smoothness.
- **Manipulability** — a scalar (Yoshikawa) that approaches zero near a kinematic singularity.
- **RNEA** — Recursive Newton-Euler Algorithm for inverse dynamics (joint torque from motion + payload).
- **Seqlock** — a lock-free reader/writer synchronisation using an odd/even sequence counter.
- **STO (Safe Torque Off)** — the hardware safety function that removes motor power independently of software.
- **TCP (Tool Center Point)** — the tool's working point; the rigid transform from the flange to that point, plus payload.
- **Protective stop** — a latched software-commanded stop (drives disabled, cause published) distinct from the hardware E-STOP.